Rapport de TP : Configuration mTLS MQTT

Auteur : Adama SANOGO **Date :** 30 Mars 2026

1. Introduction

Ce rapport détaille la mise en œuvre et la configuration de la sécurité mTLS (mutual Transport Layer Security) pour le protocole MQTT (Message Queuing Telemetry Transport) en utilisant OpenSSL et le broker Mosquitto. L'objectif principal est de sécuriser les communications entre les clients et le broker MQTT en s'assurant que les deux parties s'authentifient mutuellement à l'aide de certificats X.509. Cette approche est cruciale pour les déploiements IoT (Internet des Objets) où la sécurité et l'intégrité des données sont primordiales.

2. Concepts Fondamentaux

2.1. MQTT

MQTT est un protocole de messagerie léger, basé sur le modèle publication/abonnement, conçu pour les appareils à ressources limitées et les réseaux à faible bande passante. Il est largement utilisé dans l'IoT pour sa simplicité et son efficacité [1].

2.2. TLS/SSL

TLS (Transport Layer Security) et son prédécesseur SSL (Secure Sockets Layer) sont des protocoles cryptographiques qui fournissent une communication sécurisée sur un réseau informatique. Ils garantissent la confidentialité, l'intégrité des données et l'authentification du serveur [2].

2.3. mTLS (mutual TLS)

Le mTLS est une extension du TLS où non seulement le client vérifie l'identité du serveur, mais le serveur vérifie également l'identité du client. Cela ajoute une couche de sécurité supplémentaire en garantissant que seuls les clients autorisés peuvent se connecter au broker MQTT [3].

2.4. Certificats X.509 et Autorité de Certification (CA)

Les certificats X.509 sont des documents numériques qui lient une clé publique à une identité. Une Autorité de Certification (CA) est une entité de confiance qui émet et signe ces certificats, établissant ainsi une chaîne de confiance. Dans le cadre de ce TP, nous avons généré une CA racine pour signer les certificats du broker et des clients [4].

3. Préparation de l'environnement

Pour commencer, nous avons créé la structure de répertoires nécessaire et installé le broker Mosquitto ainsi que ses outils clients.

3.1. Création des répertoires

Les répertoires nécessaires ont été créés pour organiser les certificats et les requêtes de signature.

3.2. Installation de Mosquitto

Le broker Mosquitto et ses outils clients ont été installés sur le système.

4. Génération des Certificats et Clés

Cette section décrit la génération de la CA racine, des clés privées et des requêtes de signature de certificat (CSR) pour le broker et le client, ainsi que la signature de ces CSR par la CA.

4.1. Création de la CA Root

Une autorité de certification racine a été générée avec sa clé privée et son certificat auto-signé.

4.2. Génération des Clés Privées et CSR pour le Broker et le Client

Des clés privées ont été générées pour le broker et le client, suivies de la création de leurs requêtes de signature de certificat (CSR).

4.3. Signature des Certificats avec la CA

Les CSR du broker et du client ont été signées par la CA racine, produisant ainsi leurs certificats respectifs.

4.4. Vérification des Certificats

Les informations contenues dans les certificats générés ont été vérifiées pour s'assurer de leur conformité.

4.5. Sécurisation des Fichiers

Les permissions des fichiers de clés privées et des certificats ont été ajustées pour garantir leur sécurité.

5. Configuration de Mosquitto pour mTLS

Nous avons configuré le broker Mosquitto pour qu'il utilise les certificats générés et exige l'authentification client via mTLS.

5.1. Création du fichier de configuration Mosquitto

Un fichier de configuration spécifique a été créé pour Mosquitto, définissant le port d'écoute, les chemins des certificats et clés, et l'exigence de certificats clients.

5.2. Copie des certificats

Les certificats et clés nécessaires ont été copiés dans le répertoire de configuration de Mosquitto.

5.3. Correction des permissions et redémarrage du service

Une erreur de permission a été rencontrée lors du premier redémarrage du service Mosquitto. Cette erreur a été corrigée en ajustant les propriétaires des fichiers de clés et certificats pour l'utilisateur `mosquitto`, puis le service a été redémarré avec succès.

6. Tests de Sécurité

Nous avons effectué des tests pour vérifier que la configuration mTLS fonctionne correctement.

6.1. Publication avec certificat client (succès attendu)

Après avoir résolu un problème de vérification du nom d'hôte en ajoutant une entrée dans le fichier `/etc/hosts`, la publication d'un message MQTT avec un certificat client valide a été effectuée avec succès. Cela a démontré que le client a été authentifié correctement par le broker.

6.2. Publication sans certificat client (échec attendu)

Une tentative de publication sans fournir de certificat client a échoué, confirmant que le broker exige bien un certificat client pour établir une connexion sécurisée.

7. Conclusion

Ce TP a permis de mettre en œuvre avec succès une configuration mTLS pour un broker MQTT Mosquitto. Nous avons généré une infrastructure à clés publiques (PKI) simple avec une CA racine, des certificats pour le broker et un client, et configuré Mosquitto pour exiger l'authentification mutuelle. Les tests ont démontré que seuls les clients présentant un certificat valide et signé par la CA sont autorisés à se connecter, renforçant ainsi considérablement la sécurité des communications MQTT.

8. Références

[1] Cedalo. "MQTT TLS/SSL Configuration: Guide with Mosquitto Examples". https://cedalo.com/blog/mqtt-tls-configuration-guide/ [2] HiveMQ. "MQTT Security Fundamentals - TLS/SSL". https://www.hivemq.com/blog/mqtt-security-fundamentals-tls-ssl/ [3] NetFoundry. "IoT Authentication and MTLS". https://netfoundry.io/iot/iot-authentication-and-mtls/ [4] ThingsBoard. "X.509 Certificate Chain Authentication | MQTT Broker". https://thingsboard.io/docs/mqtt-broker/security/authentication/x509/